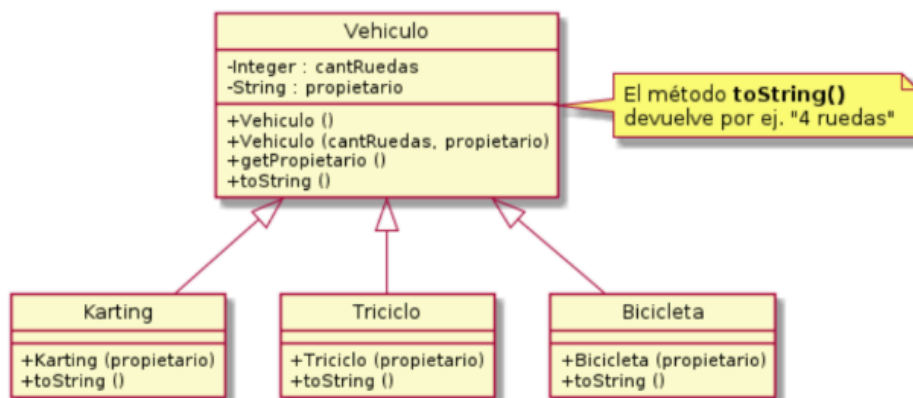


Dado el siguiente diagrama UML



Implemente el constructor y el método toString() de la subclase. Se debe generar la respuesta del ejemplo.

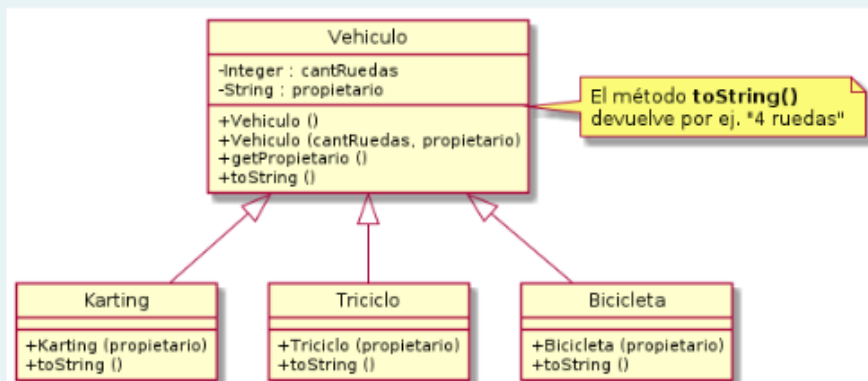
Por ejemplo:

Test	Resultado
Karting kart; kart = new Karting("Pepe"); System.out.println(kart);	Karting de Pepe (4 ruedas)

```
public class Karting extends Vehiculo {
    public Karting (String propietario){
        super(4, propietario);
    }

    public String toString(){
        return("Karting de" + this.getPropietario() + "( " +
super.toString() + " )" );
    }
}
```

Dado el siguiente diagrama UML



Implemente el constructor y el método toString() de la subclase. Se debe generar la respuesta del ejemplo.

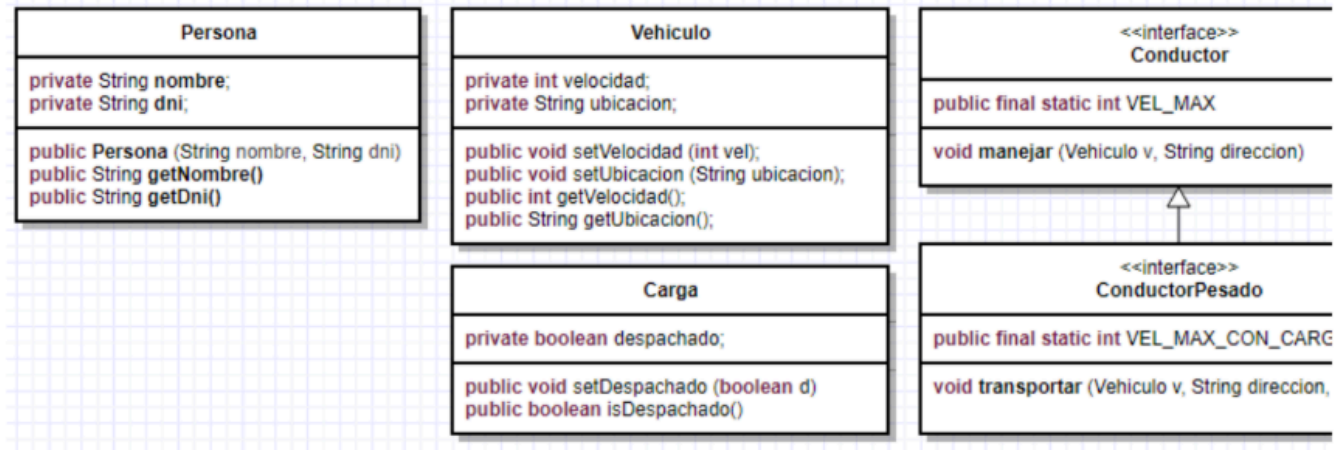
```
public class Bicicleta extends Vehiculo {
    public Bicicleta (String propietario){
        super(2, propietario);
    }
}
```

```

public String toString(){
    return("Bicicleta de" + this.getPropietario() + "( " +
super.toString() + " )" );
}
}

```

Considere las siguientes clases e interfaces



Implemente la clase **Camionero**, tal que extienda de **Persona** e implemente la interfaz **ConductorPesado** (notese que la interfaz **ConductorPesado** extiende de la interfaz **Conductor**)

(la descripción de los métodos de las interfaces se encuentran en el código)

Por ejemplo:

Test	Resultado
<pre> //Este test es solo para validar que compile Camionero camionero = new Camionero("BJ Mackey","26.178.555"); Vehiculo ab123cd = new Vehiculo(); Carga lote7 = new Carga(); camionero.manejar (ab123cd,"Av. Circunvalacion 10234"); camionero.transportar (ab123cd, "Ruta 9 km 1/2", lote7); </pre>	PASS

```

/**
 * Implemente la clase 'Camionero' que extiende de 'Persona' e
 * implementa la interfaz 'Conductor'
 *
 * 1. Debera codificar un constructor para esta clase que reciba los
 *    parametros "String nombre" y
 * 2. Debera codificar los metodos necesarios declarados en la
 *    interfaz implementada.
 * Nota: Cond extiende de Conductor
 * 3. Un camionero maneja a la maxima velocidad permitida para un
 *    Conductor (VEL_MAX), pero tran
 * la velocidad maxima permitida para circular con carga
 * (VEL_MAX_CON_CARGA)
 */

```

```

* NOTA: Segun nuestro modelo, asuma los siguientes comportamientos
para los metodos declarados
* void manejar (Vehiculo v, String direccion)
* a. Llevar el vehiculo a una direccion, (i.e., setea la direccion
en el vehiculo)
* b. Setear una velocidad crucero.
*
* void transportar (Vehiculo v, String direccion, Carga c):
* a. Llevar el vehiculo a una direccion.
* b. Setear una velocidad crucero.
* c. Marcar la carga como "despachada"
**/

```

```

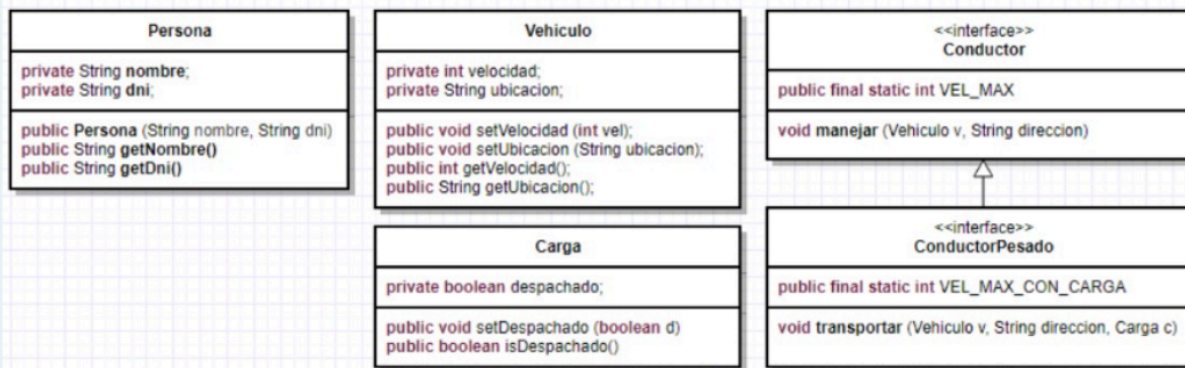
public class Camionero extends Persona implements ConductorPesado(){
// Constructor que recibe nombre y dni, delegando la inicialización
// a la clase padre Persona.
    public Camionero(String nombre, String dni){
        super(nombre, dni);
    }

//Lleva el vehículo a una direccion a la velocidad maxima permitida
// para un Conductor comun (VEL_MAX).
    void manejar (Vehiculo v, String direccion){
        v.setUbicacion(direccion);
        v.setVelocidad(VEL_MAX);
    }

//Lleva el vehículo a una direccion, a velocidad crucero con carga.
//Marca la carga como despachada.
    void transportar (Vehiculo v, String direccion, Carga c){
        v.setUbicacion(direccion);
        v.setVelocidad(VEL_MAX_CON_CARGA);
        c.setDespachado(true);
    }
}

```

Considere las siguientes clases e interfaces



Implemente la clase **Taxista**, tal que extienda de **Persona** e implemente la interfaz **Conductor**.

(la descripción de los métodos de las interfaces se encuentran en el código)

Por ejemplo:

Test	Resultado
<pre>//Este test es solo para validar que compile Taxista taxista = new Taxista("James Bond","007"); Vehiculo ab123cd = new Vehiculo(); taxista.manejar (ab123cd,"Av.Colon 1234");</pre>	PASS

```

public class Taxista extends Persona implements Conductor {

    /**
     * Constructor que recibe nombre y dni,
     * delegando la inicializacion a la clase padre Persona.
     */
    public Taxista(String nombre, String dni) {
        super(nombre, dni);
    }

    /**
     * Lleva el vehiculo a una direccion, a la velocidad
     * maxima permitida para un Conductor (VEL_MAX)
     */
    public void manejar(Vehiculo v, String direccion) {
        v.setUbicacion(direccion);
        v.setVelocidad(VEL_MAX);
    }
}
  
```

Considere el siguiente diagrama de clases, en donde la clase **ServidorDeReservas** representa un servidos que gestiona reservas en distintas aerolineas, y la clase **Agencia** es un cliente que utiliza este servicio para comprar pasajes.



Implemente el metodo comprarPasaje de la clase Agencia, segun la documentacion del metodos.

```

public class Agencia {
    public static final String ERROR_NO_DISPONIBLE = "Error No
    Disponible. Mensaje: ";
    public static final String ERROR_INESPERADO = "Error Inesperado.
    Mensaje: ";
    private ServidorDeReservas servidor;
    public Agencia (){
        servidor = new ServidorDeReservas();
    }

    /* Compra un pasaje en el vuelo especificado, en la fila y
    ubicacion *pasados como argumento y retorna un Strin con el codigo
    de reserva *retornado por el metodo "reservar" *del
    ServidorDeReservas.
    *Si se produce una Excepcion por no estar disponible el vuelo /
    *asiento, retorna un String con el texto ERROR_NO_DISPONIBLE y el
    *mensaje de la excepcion. Si se produce un error inesperado retorna
    *un String con el texto ERROR_INESPERADO y el mensaje de la
    *excepcion.
    */

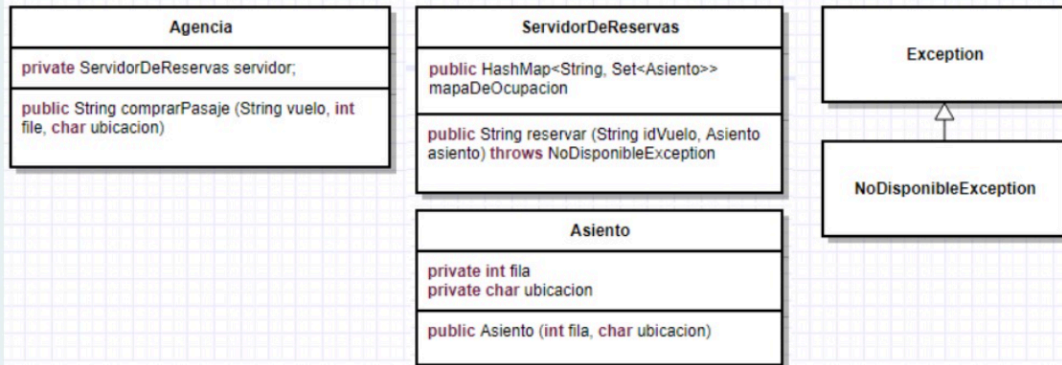
    public String comprarPasaje(String idVuelo, int fila, char
    ubicacion){
        try{
            String cadenaDeConfirmacion;
            Asiento a = new Asiento(fila, ubicacion);
            cadenaDeConfirmacion = servidor.reservar(idVuelo, a);
        }
        catch(NoDisponibleException e){
            return ERROR_NO_DISPONIBLE +e.getMessage();
        }
        catch(Exception e){
            return ERROR_INESPERADO + e.getMessage();
        }
    }
  
```

```

    }
}

```

Considere el siguiente diagrama de clases, en donde la clase **ServidorDeReservas** representa un servidor que gestiona reservas en distintas aerolíneas, y la clase **Agencia** es un cliente que utiliza este servicio para comprar pasajes.



Implemente el método **reservar** de la clase **ServidorDeReserva**, según la documentación del método.

```

import java.util.HashMap;
import java.util.Set;

public class ServidorDeReservas {
    public static final String ARGUMENTOS_INVALIDOS= "Parametros Invalidos";
    public static final String VUELO_DESCONOCIDO = "Vuelo Desconocido";
    public static final String ASIENTO_OCUPADO = "Asiento Ocupado";
    public static final String PREFIJO_CONFIRMACION = "Conf-";

    //Mapa de ocupacion. Asocia un vuelo (idVuelo) con un Set<Asiento> que
    //contiene el conjunto de asientos ocupados.
    public HashMap<String,Set<Asiento>> mapaDeOcupacion;

    public ServidorDeReservas() {
        mapaDeOcupacion = new HashMap<String,Set<Asiento>>();
    }

    public String reservar(String idVuelo, Asiento asiento)
    throws NoDisponibleException {

        // 1. Validar argumentos null
        if (idVuelo == null || asiento == null) {
            throw new
IllegalArgumentException(ARGUMENTOS_INVALIDOS);
        }
    }
}

```

```

// 2. Validar que el vuelo exista en el mapa
if (!mapaDeOcupacion.containsKey(idVuelo)) {
    throw new NoDisponibleException(VUELO_DESCONOCIDO);
}

// 3. Obtener los asientos ocupados de ese vuelo
Set<Asiento> asientosOcupados =
mapaDeOcupacion.get(idVuelo);

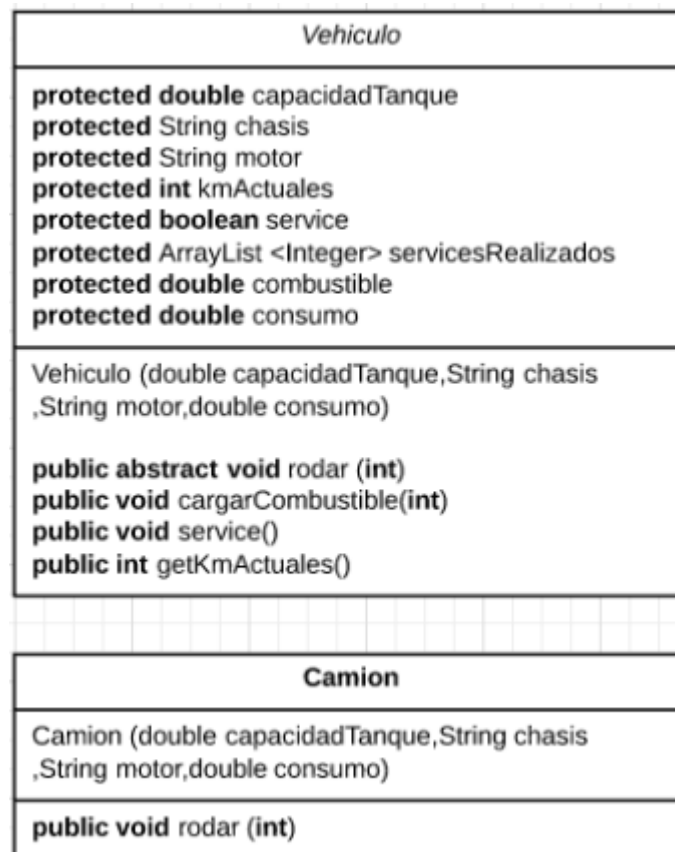
// 4. Validar que el asiento no esté ocupado
if (asientosOcupados.contains(asiento)) {
    throw new NoDisponibleException(ASIENTO_OCUPADO);
}

// 5. Marcar el asiento como ocupado
asientosOcupados.add(asiento);

// 6. Construir y devolver el código de confirmación
return PREFIJO_CONFIRMACION + idVuelo + "-" +
asiento.getFila() + asiento.getUbicacion();
}
}

```

Dadas las clases:



Implemente la clase Camion que herede de la clase abstracta Vehiculo.

```
public class Camion extends Vehiculo{
//constructor con parametros
public Camion (int capacidadTanque, String chasis, String motor,
double consumo){
    super(capacidadTanque, chasis, motor, consumo);
}

/*Implemente los metodos faltantes- "rodar" incrementa los kms
*actuales del vehiculo, y descuenta la cantidad consumida (consumo
* *km) del combustible del vehiculo.
*Si la cantidad de combustible del vehiculo es insuficiente para
*recorrer la cantidad de kms solicitados, no se actualiza ninguna
*variable.
*/

public void rodar (int km){
    if (consumo * km <= km){
        kmActuales += km;
        combustible -= consumo * km;
    }
}
}
```

Dada la siguiente interface y el siguiente enumerado:

```
public interface Edificio{
double getSuperficieEdificio();
}
```

Modifique la clase Oficinas para que implemente la interface Edificio. Considere el método:

* public double getSuperficieEdificio() ----> Retorna el area del edificio considerando el total de todos los pisos*

```
public class Oficinas implements Edificio{
private int numeroDePisos;
private double ancho;
private double largo;
/**
* Constructor con parametros
* @param numeroDePisos la cantidad de pisos del inmueble
* @param ancho el ancho del edificio
* @param largo el largo del edificio
*/
    public Oficinas(int numeroDePisos, double ancho, double
largo){
        this.numeroDePisos = numeroDePisos;
        this.ancho = ancho;
        this.largo = largo;
    }
}
```



```

    }

    public double getSuperficieEdificio(){
        return this.numeroDePisos*this.ancho*this.largo;
    }
}

```

Dada la siguiente interface y el siguiente enumerado:

```

public interface InstalacionDeportiva{
double getSuperficie(); }

```

Implemente la clase Polideportivo que implementa la interface InstalacionDeportiva.

Considere el metodo * public double getSuperficie() ----> Devuelve la superficie del polideportivo*

```

public class Polideportivo implement InstalacionDeportiva {
    private double largo;
    private double ancho;
    private String nombre;
    public Polideportivo(double largo, double ancho, String nombre){
        this.largo = largo;
        this.ancho = ancho;
        this.nombre = nombre;
    }

    public double getSuperficie(){
        return ancho * largo;
    }
}

```